

# Real-Time Visualization of Digital Modulation and Channel Impairments Using a Single PlutoSDR

Speaker Script | Howard Wang | ENGR 78 Final Project | Spring 2026

---

## Slide 1 – Title

---

Hi everyone. My project is a real-time visualization tool for digital modulation and channel impairments using a single ADALM-PLUTO software-defined radio. The basic idea is simple: instead of only calculating bit error rate or looking at static textbook plots, this system lets you transmit, receive, impair, synchronize, and visualize a digital communications link live. I will walk through the problem, the system architecture, what each visualization means, how the receiver handles synchronization, and what I observed from both simulation and PlutoSDR loopback testing.

## Slide 2 – The Problem

---

The problem I wanted to solve is the gap between equations and intuition. In class, we learn formulas for BER, SNR, modulation, and pulse shaping. Those formulas matter, but they do not always make the behavior feel real. For example, I can calculate the BER for BPSK in AWGN, but that does not automatically tell me what the constellation should look like as the noise increases, or what happens to an eye diagram when multipath creates inter-symbol interference. So the goal of this project was to make those effects visible. I wanted a tool where changing a channel parameter immediately changes the signal displays, so students can connect the math to the actual shape of the received waveform.

## Slide 3 – What We Built

---

What I built is an interactive digital communications workbench in Python. It can generate payload bits, map them into modulation symbols, pulse-shape them, send them through an ADALM-PLUTO SDR, and receive them back either through RF loopback or through a full software simulation path. After reception, the system can inject controlled impairments such as noise, phase offset, frequency offset, fading, and multipath. The GUI shows five live views: the spectrum, constellation, eye diagram, rolling metrics, and throughput. There is also a BER sweep mode that automatically tests different SNR values and compares measured BER against theory. So the project is not just a plotter. It is a complete transmit, receive, impair, synchronize, demodulate, and measure pipeline.

## Slide 4 – System Architecture

---

This slide shows the full signal path. Read it from left to right, starting on the top row. The transmit chain starts with payload bits, maps them into symbols, pulse-shapes them with a root-raised-cosine filter, builds a frame with a preamble and payload, and sends the samples through the PlutoSDR transmitter. The RF output is connected back to the receiver through an SMA loopback cable and a 30 dB attenuator, so the same Pluto device acts as both transmitter and receiver. On the receive side, the samples can go through software impairments, then synchronization, demodulation, and metric computation. The PyQt GUI reads the latest buffers and metrics from the DSP pipeline and updates the plots in real time.

## Slide 5 – How to Read the Architecture

---

This slide breaks down what each block means. On the transmit side, the payload bits are just random test data, which lets the receiver compare transmitted and recovered bits later. The symbol mapping block turns those bits into points in a constellation, such as BPSK, QPSK, 8-PSK, or 16-QAM. The root-raised-cosine filter limits the signal bandwidth and reduces inter-symbol interference. The frame builder adds structure around the payload, especially the preamble, so the receiver can find the beginning of each frame and estimate offsets. On the receive side, the Pluto loopback gives us real RF hardware behavior, while the software impairment block gives us repeatable control over noise and distortion. Synchronization then fixes timing, phase, and frequency errors before demodulation. The important design choice is that impairments are applied after RX, so sliders respond instantly and both hardware and simulation use the same downstream DSP and GUI path.

## Slide 6 – Four Modulation Schemes

---

The system supports four modulation schemes: BPSK, QPSK, 8-PSK, and 16-QAM. BPSK carries one bit per symbol, QPSK carries two, 8-PSK carries three, and 16-QAM carries four. That means higher-order schemes improve data rate, but the constellation points are closer together, so they are more vulnerable to noise and distortion. All of the mappings are Gray-coded, which means neighboring symbols differ by only one bit. That reduces the number of bit errors when the receiver chooses a nearby wrong symbol. The signal is pulse-shaped with a root-raised-cosine filter using roll-off 0.35 and 8 samples per symbol. On the receiver side, demodulation uses nearest-neighbor hard decisions, which means each received symbol is assigned to the closest ideal constellation point.

## Slide 7 – Channel Impairments – See the Damage

---

This slide shows the main channel impairments and how each one damages the received signal. AWGN spreads each ideal constellation point into a cloud, and the cloud gets wider as SNR decreases. A phase offset rotates the entire constellation by a fixed angle. A frequency offset is worse because the constellation keeps rotating over time unless the receiver estimates and corrects it. Multipath creates delayed copies of the signal, which smear symbols into each other and cause inter-symbol interference. Flat fading scales the amplitude, so the constellation can shrink or grow depending on the channel gain. The point of the live GUI is that these effects are not abstract anymore. You move a slider and immediately see which part of the signal representation breaks first.

## Slide 8 – Receiver Synchronization

---

Synchronization is one of the most important parts of the project. If the receiver does not know where a frame starts, what the frequency offset is, what the phase offset is, or where the best symbol sampling instant is, the demodulator will fail even if the modulation code is correct. The frame begins with a Barker-13 preamble, which has good correlation properties, so the receiver can detect the frame start by cross-correlating against a known template. Repeated preamble sections help estimate frequency offset from phase differences over time. Pilot symbols help estimate and correct residual phase error. Then Gardner timing recovery adjusts the sampling instant so symbols are sampled near the center of the eye opening. This chain is what turns a noisy received waveform back into symbol decisions.

## Slide 9 – Eye Diagram – Reading Signal Health

---

The eye diagram is a compact way to judge signal quality. It overlays many short segments of the matched-filtered waveform on top of each other, usually over one or two symbol periods. When the eye is open, the receiver has a clear decision instant and the symbol levels are well separated. When noise increases, the traces thicken vertically. When timing jitter increases, they smear horizontally. When multipath or filtering problems introduce inter-symbol interference, the eye starts closing because neighboring symbols affect the current one. So the eye diagram is useful because it shows several problems at once: noise margin, timing margin, and ISI. In this project, it complements the constellation view because the constellation shows symbol decisions while the eye shows time-domain health.

## Slide 10 – Live Mode – GUI Walkthrough

---

This is the main live interface. The spectrum panel shows the received signal in the frequency domain, so we can see bandwidth, noise floor, and spectral shape. The constellation panel shows received symbols against the ideal reference points, which makes noise, rotation, scaling, and clustering easy to see. The eye diagram shows time-domain signal health after matched filtering. The metrics panel tracks BER, EVM, and estimated SNR over time, so the display is not only visual but also quantitative. The throughput and cumulative-bits panel tells us how much payload is actually being decoded. The control panel lets the user change modulation and impairments while the system is running, which is the main educational value of the tool.

## Slide 11 – BER Sweep Mode – Theory Meets Measurement

---

The BER sweep mode is where the project connects the live visualization back to communications theory. Instead of manually dragging the noise slider, the system steps through a range of SNR values, transmits or simulates frames, demodulates them, counts bit errors, and plots measured BER. The dashed lines are the theoretical curves. For BPSK and

QPSK, the reference is based on the Q-function expression for AWGN. For 16-QAM, the reference uses the standard erfc approximation. If the measured points line up with theory, that means the modulation, impairment model, synchronization, and demodulation pipeline are behaving correctly. In hardware loopback, the curve can shift by about 0.5 to 1 dB because real SDRs add effects like quantization noise, DC offset, and I/Q imbalance.

## Slide 12 – Throughput & Cumulative Bits

---

This slide focuses on data flow rather than only error rate. Throughput is the number of decoded payload bits per second. For QPSK at around 20 frames per second, the system reaches about 40 kilobits per second. For 16-QAM, because each symbol carries twice as many bits as QPSK, the throughput is closer to 80 kilobits per second under similar conditions. The exact value depends on USB timing, synchronization success, and how many frames are being processed. The cumulative-bits trace is useful because it shows whether the link is continuously delivering data. A healthy link gives a steady ramp upward. If synchronization fails or BER becomes too high, the ramp slows down or flattens. That makes link reliability visible over time.

## Slide 13 – Software Architecture

---

The code is organized into modules that match the signal chain. The modulation module handles constellation definitions, Gray-coded bit mapping, demodulation, and root-raised-cosine pulse shaping. The impairments module applies AWGN, phase offset, frequency offset, fading, and multipath. The synchronization module handles preamble detection, frequency correction, phase correction, and Gardner timing recovery. The Pluto interface wraps the SDR hardware access and includes USB detection. The DSP worker runs the signal processing pipeline in a background QThread. The GUI uses PyQt6 and PyQtGraph to render the plots. One important architecture decision is that the DSP worker writes only the latest results to shared memory, while the GUI polls that state on a timer. That avoids a queue backlog and keeps the interface responsive.

## Slide 14 – Demo Scenarios – Try It Yourself

---

These are the demo scenarios that best show the system. The first is noise floor exploration: start at a clean SNR and lower it until the constellation spreads out and BER rises. The second is multipath: add a delayed copy of the signal and watch the eye close because symbols interfere with neighboring symbols. The third is phase and frequency offset: a fixed phase offset rotates the constellation, while a frequency offset makes the constellation rotate continuously until synchronization corrects it. The fourth is scheme comparison: keep the channel fixed and switch between BPSK, QPSK, 8-PSK, and 16-QAM. That makes the tradeoff obvious. Higher-order modulation gives more bits per symbol, but the smaller symbol spacing makes it less robust.

## Slide 15 – Results & Observations

---

The main quantitative result is that the measured BER curves match theoretical AWGN predictions within less than about 1 dB. That is important because it validates more than just one formula. It validates the full chain: frame construction, pulse shaping, channel model, synchronization, demodulation, and error counting. In hardware loopback, the implementation loss is roughly 0.5 to 1 dB, which is reasonable for a low-cost SDR setup. The Gardner timing recovery locks within about 2 to 3 symbols when conditions are reasonable. In terms of performance, simulation mode reaches about 20 frames per second with roughly 6 milliseconds of DSP work per frame. Hardware mode is closer to 15 frames per second because USB transfer latency becomes part of the loop.

## Slide 16 – Lessons Learned

---

The biggest lesson is that synchronization is harder than modulation. Creating a constellation and mapping bits to symbols is fairly direct. Making the receiver recover frame start, frequency offset, phase offset, and timing under real impairments is where most of the work is. The second lesson is that GUI design matters for DSP tools. My first approach pushed every processed buffer into the Qt event queue, which caused lag. The better design was a worker thread with shared latest-state data and a GUI timer. The third lesson is that hardware is never as clean as simulation. The PlutoSDR adds latency, DC offset, I/Q imbalance, and other nonideal behavior. Finally, the visualization itself really helps intuition. Watching the signal degrade makes the equations easier to believe.

## Slide 17 – Conclusion

---

To conclude, this project built a complete real-time digital communications visualizer. It supports four modulation schemes, multiple controlled impairments, a full receiver synchronization chain, live spectrum, constellation, eye, metrics, throughput, and cumulative-bit displays, plus a BER sweep mode for comparison with theory. It can run with a real PlutoSDR loopback or in simulation, so it works both as a hardware experiment and as a classroom demonstration tool. The broader point is that digital communications becomes much easier to understand when the math is connected to moving signals and live measurements. Thank you, and I am happy to take questions.